R packages development training

Mathieu Depetris

Table of contents

Welcome!

Welcome to the online support to learn how to organise your R code through the creation of an R package. You will find that this process is easier than you can imagine and with a little bit of pratiques you will be able to create your own package. In addition, you will discover that this way open you the field to develop reproducible code, documented faster and simply your processes at many levels and learn how other people can download and use it.

 At least you will find below several training sessions available if you want a guided explanation for R package creation.

To conclude, this support was developed according to the book *R Packages (2e)* [1] written by Hadley Wickham and Jennifer Bryan. An online version is available through this link, don't hesitate to take a look.

Introduction

To do

Part I

Training framework and requirements

1 - Operating system and working environment associated

Before going into the fun of R package development, take a moment to setup our working environment and overview some interesting software which can simplify our life.

First, all the training and test that we will do among this book will be executed with Windows Operating System (OS). When you develop your package, one of the aims could be to share or propose your work to other peoples and you have to take into account about the variability of the processes among OS, software for example R dependencies. We will put in place several processes to support us in our task but keep in your mind that processes that work on an OS could not work in the same pattern in another one. Futermore, one of our tasks among the lifecycle of our package will be to maintain a maximum of compatibility, for example through integration of update or evolution in our code.

Developing R packages assume that you have a little experience with the R software and not start from scratch with it. Even if you see that R package development is not so hard that it looks like, you will find a lot of resources on the internet to support you if you want to start with R and develop your skills in it.

1.1 Requirements software

For the training we will use the *R* version 4.5.0 for Windows available through the web page of The Comprehensive R Archive Network. You will find also precompiled binary distributions for Linux and macOS.

In addition, we will use the Integrated Development Environment (IDE) *RStudio*. Today this software is one of the most used in the R communities and includes a lot of things like a console, syntax-highlighting editor that supports direct code execution, and tools for plotting, history, debugging, and workspace management. Furthermore, recent evolution like the creation of Posit compagny make the software ecosystem more friendly connected with R and Python. For the training we will use the RStudio version 2025.05.0+496 "Mariposa Orchid" for Windows OS. You can download the one for your configuration throught the Posit download webpage.

At least, we will using Git, for version control system, to host our R package and apply several processes of Continuous Integration (CI), most of them already prepare by the R communities. Today, this is a very precious tool that helps developers track changes in their code, collaborate efficiently, and manage different versions of a project. During this training we will use one of them. You can download the software on the offical website throught this link.

> **!** Important
>
> Under Windows OS, it's necessary to install the RTools software. This is a toolchains for building R and R packages from source on Windows. And don't need it on macOS and Linux because theses OS have usually pre-installed compilation tools. You can download it from the offical link.Be aware that you need to take the correct version according to our R version installated (for this training it would be the RTools 4.5).

1.2 Additional software

Several software are not necessary to create your own R package but bring you a lot of improvement and at leave simplify you life.

The first one is Visual Studio Code (or VS Code) which is a free, lightweight, extensible code editor developed by Microsoft. He supports a wide variety of programming languages such as R, JavaScript, Python, C++, Java, HTML/CSS. In addition is cross-platform compatibility, support Git and other version control systems and one of his best advantages is to provide extensions to add functionality and customize the environment. If you want you can download it from this offical website and if it's possible according to your configuration prefer a system installer version. For this training I will use the version 1.100.2 on Windows OS.

According to the use of Git, we will use one of them call GitHub. You will see that a loot of processes were developed for work natively with GitHub and facilitate your R package creation. If you want to use another one, like a GitLab hosted by your company for example, you can of

course but you will have to modify and adapted by yourself several processes propose during this training. In addition we will use here a Graphical User Interface (GUI) for managing Git repositories called GitHub Desktop. This is a free, open-source application which simplifies current Git operations, making it easier for developers to clone repositories, commit changes, create branches, and push updates without using the command line. Like its name suggests is working natively with GitHub repositories but you can also use it for connection with other Git like GitLab. Officially is available for Windows and MacOS but you can also find adapted version for Linux on the internet. For download it goes through the following link.

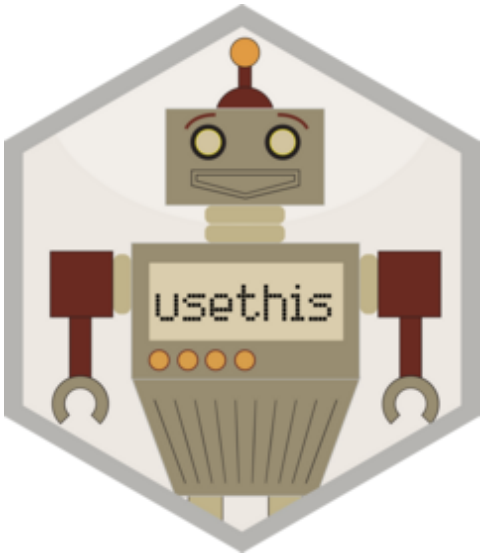
2 - Overview of R packages used

This section aim to introduce brively which R packages environment we will use. Since several years, the R communities evolve in many interesting ways and today if you want to write your own package there are a lot of support that you can find.

First of all, it's difficult to talk about package whithout talk about the tidyverse approach. Tidyverse is a collection of R packages designed for data science. They share an underlying design philosophy, grammar, and data structures. If you use R, it's allmost sure that you have allready use this kind of packages, maybe even without knowing them. Even if there are none mandatary for R use, theses packages bring a lot of benefices and at the end make data science easier, faster, and more fun (yes, you will see).

So far we don't go deeper into all the useful R packages of the tidyverse collection, but only focusing on theses related to the R packages developement.

The first one is called devtools [2]. The aim of this package is to make R package development easier by providing R functions that simplify and expedite common tasks. In practical, if you use the Rstudio IDE you will not interact directly with this package because the majority of common processes will be integrated automatically in the Rstudio environment and become "click button" processes.



The second one is the sub-package you are most likely to interact with directly. It be called usethis [3] and is aim is to automates repetitive tasks that arise during project setup and development. The idea is to, if it's possible, use functions available by the package, rather that modify directely the code, to reduce error and be sure to have the last version of process strucutre.

There are two more important packages related to design of the documentation: roxygen2 [4] and pkgdown [5]. With the first one throught the use of tags, the code and the documentation are adjacent and dynamically inspects the objects that it's documenting. We will go back to them into the following sections but keep in your mind that theses two tools simplify and automate the documentation creation and evolution.

The last one is called testthat [6] and focusing on the field of testing your code, which can be painful and tedious, but it greatly increases the quality of your code.

In addition we will use or talk about several other packages, for example lintr [7] or renv [8], but it's more a case-by-case utilisation and we will discuss them when the time comes.

3 - Before go down the rabbit hole

Before we start to developing our first R package, let's talk briefly about the aim to organise your code into a package and in fact why we consume time to develop this kind of structure.

When you use R, a package is the fundamental unit of shareable codes and brings a lot of advantage, like regroup in one place all the stuff (code, data, documentation for example), improve the standardisation of your process and the sharing them easier. In fact, build an R package is nothing more than a way of organising things that you have already done by another way and facilitate at the end the life cycle of your processes.

Furthermore, you will see quickly that you can use a lot of processes already developed ,and update in time, by the R communauties. There are package that make package, processes that allow to automated tasks and the tidyverse philosophy explain in the previous section help a lot.

To summarise:

- don't worry, you will see that all this process is painless and you will be able to create very great thing in brief time,
- in addition, you will save time! you will follow a template and automate a lot of processes with the rule "anything that can be automated, should be automated",
- to conclude, you will use standardised conventions that open to you the path a several free and evolving standardised tools.

Part II

First part - Global package architecture

4 - Before create anything

Before starting playing with functions and create the global structure of your package, the best thing to do is to think a little bit about what you want to create. The idea is not to spend a very long time to do that, but if you take this time (especially is it's the first time) you will see that at the end you will gain. The majority of the package component can be modified after definition, but some update could be heavy and painful and sometimes it's simple to avoid that by taking a moment of reflection.

4.1 First reflexion, the perimeter

The aim here is to answer the question, what I want to put into my package or what I want my package will do? You don't have to be exhaustive (if you can it's better) but try to explore several fields like:

- If I have several functions already developed (even partially), it's logical to put all of them into a single structure?
- If my package will be used by other users, does the process of use if relevant and logical for users?
- If I think bigger, do you think my package could be placed in specific environment and maybe interact with other packages? A good example of that is if a package interact with other packages and each one has a specific purpose (for example one dedicates to the controls and another dedicates to the graphic display).

You can image other cases and again it's normal if your perimeter evolve with time regarding the maturity of your work. A good solution could be to design flux diagrams for example explain (even for you) the process and maybe associated one. In addition, these kinds of figures could be useful for the documentation process. A quick research on the internet will be displaying several software, free and paid, for doing that.

4.2 Second reflexion, the name

Now you know globally the purpose of my package. The next step is to define is named. It's an important moment because the name is one of the first things that you and the others will see. At a time where the referencing (like the SEO or Search Engine Optimisation) and visibility is

important items regarding the life of your work, it could be bad to lose points at this moment just because the name is not very clear and friendly.

Globably, I advise you to follow several good praticals:

- the name has to be **simple** and **clearness**. What's better than a name that's easy to understand and remember and ideally allows a reading to understand what it does.
- avoid **special caracters**, like the accents or dot, and prefer use convention like Snake Case (each word is separated by and underscore character) or at least Camel Case (start with the first word with lowercase and capitalize the first letter of each word that follows withou space).
- insure the **compatibilty** of the name with package respositories like the CRAN (Comprehensive R Archive Network) or Bioconductor.

To support you in this step, you can use functions of the R package, pak [9].

This package is a modern alternative to the R standard function, `install.packages()`. Is goal is make package installation faster and more reliable. In particular, it performs all HTTP operations in parallel, so metadata resolution and package downloads are fast. In addition, he has a dependency solver, so it finds version conflicts before performing the installation. Futhermore, he supports CRAN, 'Bioconductor' and 'GitHub' packages as well.

```
package_name <- "lightning"
library(pak)
pkg_name_check(name = package_name)
```

```
                              -*- lightning -*-

   valid name         CRAN               Bioconductor       not a profanity

 Wikipedia
 Lightning Lightning is a natural phenomenon consisting of electrostatic
 discharges occurring through the atmosphere between two electrically
 charged regions. One or both regions are within the atmosphere, with the
 second region sometimes occurring on the ground. Following the lightning,
 the regions become partially or wholly electrically neutralized.

 …
                      https://en.wikipedia.org/wiki/Lightning
 Wiktionary
 lightning Etymology: From light(e)n + -ing.  Doublet of lightening.
 Noun: lightning (usually uncountable, plural lightnings)
```

```
A flash of light produced by short-duration, high-voltage discharge of
electricity within a cloud, between clouds, or between a cloud and the
earth.
A discharge of this kind.
…
                    https://en.wiktionary.org/wiki/lightning

Sentiment: :| (0)
```

4.3 Third reflexion, the logo

Even if it's not mandatory, what is more fun than you have your own package logo. If you are familiar with the tidyverse R package communities, you should see that each package has its own logo with some similar structure of hex stickers.

If you want to design your own logo:

- the convention use for tidyverse package is to orient the logo with a vertex at the top and bottom, with flat vertical sides.
- take of look on the hexbin community website when you have a lot of hexagon stickers and some recommendation for the "best sticker size" (figure 1).

Figure 1: Standard dimensions of a hex sticker

- The hexSticker package [10] helps you make your logo from within the comfort of R.
- Another good solution is to use an AI image generator. Without open the field of which one you have to select, according to several considerations like the integrity of the processes and the ecological impact of that, this kind of software be able to generate your logo easily according to your preference.

If you have one at the moment, keep it closely and we will see how to integrate it in our package structure. If not don't worry, you can go back to this process further.

5 - Creation of the package structure

5.1 - Initiation the global architecture

Now you have an working environment correctly configured and we have taken a few moment to thing about our package aim. We can go deeper and initiate the structure of our new package from an active R session. You don't have to take care about the configuration of your local session because the function associated will be create a clean new project for your package. First, load the devtools and by association is friend usethis.

```
library(devtools)
```

```
Loading required package: usethis
```

Just to be sure that you have everything you need to develop R package you can launch the following command line to display a report of the package development situation.

```
dev_sitrep()
```

```
-- R -----------------------------------------------------------------------
* version: 4.5.2
* path: '/opt/R/4.5.2/lib/R/'
-- devtools ----------------------------------------------------------------
* version: 2.4.6
-- dev package -------------------------------------------------------------
* package: <unset>
* path: <unset>

v All checks passed
```

If everything rock's, launch the next command to initiate the global architecture of your package. For the training, my new R package will have the acronym "macfly" for "My Anwesone paCkage For Learning with Yearning".

```r
create_package(path = path.expand("~/macfly"),
               rstudio = TRUE,
               open = FALSE)
```

```
v Setting active project to "/home/runner/macfly".
```

```
Package: macfly
Title: What the Package Does (One Line, Title Case)
Version: 0.0.0.9000
Authors@R (parsed):
    * First Last <first.last@example.com> [aut, cre]
Description: What the package does (one paragraph).
License: `use_mit_license()`, `use_gpl3_license()` or friends to pick a
    license
Encoding: UTF-8
Roxygen: list(markdown = TRUE)
RoxygenNote: 7.3.3
```

```
v Setting active project to "<no active project>".
```

Another solution with RStudio is to use the interface through the *file* menu, *New Project…*, *New Directory*, *R Package*, file the field "Package name:", click on "Open in new session" and finish with *Create Project". Even is the approach is similar, the click-button interface create in addition a template function with a proposal for the documentation associated. In opposite, the command line creation include by default a .gitignore file (for Git use). It's not true important but it could be a bit better to use the command line because we will learn how to create a R function template later and in your developer life it could be very rare (and dangerous) when you don't use a Git associated to your work.

By default, RStudio open a fresh session associated to the R environment of your package and you should need to call again `library(devtools)`.

> **Note**
>
> For your information, you will find in the usethis package a function `create_tidy_package()` that extends `create_package()` by applying many of the tidyverse development conventions as possible. I don't recommend using that way, instead of you know exactly what you do and to save time in the future, because it's better to really understand each step of the package creation.

> **!** Important
>
> At least, you could see sometime a function call `package.skeleton` from the utils R package. Even if this function create a package structure it's an "old way" to do and you should have some trouble if you use it, especially with you want to use the tidyverse philosophy.

5.2 - Overview of the package strucutre

If you follow the previous step, you should have the opening of a new RStudio session and the creation of a new directory on your local drive called just like your package name. Let take a moment that.

5.2.1 - The package directory

Inside the package directory, you should see 8 elements (the `.Rhistory` file is normally generated when you close for the first time in your RStudio session, but his generation depends on your personal configuration).

> **i** Note
>
> If you show a number or kinds of elements different than below without change anything in the command line below, maybe it's because you don't have to unlock the hidden files. In order to remedy you can go in the setting directory option (normaly and can access to that directely in any directory on Windows) and in the parameters apply the visiblity of hidden elements. In RStudio to see hidden files in the *files* panel, go to *More* (gear symbol) the tick *Show Hidden Files*.
> In addition, if highly recommend displaying file extension in your file explorer (on Windows, it's normally near the option for hidden elements). By default this display is hidden and could be a little bit strange when you activated it, but it's very important to sensitize the user of what kind of file you will open. Furthermore, be aware the "clothes don't make the man" and the extension is only a shortcut for the system to know how to deal with that but don't make any guarantee of what you have inside. In accordance with rules of good practice (especially link to the security), take a moment to think about what you're about to open and display the extension is a good first step to begging.

Element name	Type
.Rbuildignore	file
.gitignore	file

Element name	Type
DESCRIPTION	file
NAMESPACE	file
R	directory
macfly.Rproj	file

- *.gitignore*, we will see in the next step (put here section id) how to connect our local package directory to a git. This file indicates which files or directories have to be ignored for the git commits.
- *.Rbuildignore*, in the same way that the *.gitignore*, this file indicate which elements have to be excluded during the package compilation.
- *.Rhistory*, by default this file keeps tracks of every command you've typed into the R console. When you close R, it often saves this file automatically. Be aware of that process, because this file may contain sensitive data (access to local files, credentials, etc.). You will see later that when we make connection with Git, we will ignore this file through the *.gitignore* file. If you don't want to generate and keep this file, in RStudio *Global Options*, section *General* you can disable "Always save history (even when not saving .RData)".
- *.Rproj.user*, this directory is normaly hidden and used internally by RStudio.
- *DESCRIPTION*, this is an important file that provides metadata about your package, like the authors, the package description and the dependencies.
- *macfly.Rproj*, this file, dedicate to RStudio, aims to organize your work through define the working directory of your project, memorize parameters specifics and able to open directly the project in RStudio by double click on it.
- *NAMESPACE*, this file aims to declares functions "used" among your package (through external and internal ways).
- *R*, this directory will contains all the functions developed in your package.

5.3 - RStudio package environment

If you take a look at the panel *Build* in the Rstudio environment you should see several new sections, like *Install* or *Check* (figure 1). We will take a look after in each of one, but keep in our mind that this sections is a shorcut way to apply processes on your package.

Figure 1: RStudio Build panel

> **Note**
>
> In preparation for the documentation generation, the only thing to do now is to go to the *Configure Build Tools* options (click on *More* (gear symbol) and *Configure Build Tools*). In the *Project Options* menu, *Build Tools* section, click on the *Configure* button near *Generate document with Roxygen*. In the new windows, fill all the proposals. This

process will create and update documentation automatically when you launch several packages processes. Sometimes, it may justify not filled all is if you have a really huge documentation or a specific rule for documentation generations.

6 - Package states

Now we have created our package structure, it's important to take a moment to understand exactly the different states that your package can have and in addition make a link with all the things that you already do, like use the `install.packages()` function, maybe without knowing what is behind that. There are five different package states that we will explain below.

6.1 - Source package

When we create our package in section 5, we create a source structure package. This is not the comment form you are most familiar with but to summarize it's just a directory with files and subdirectories, grouped in a specific structure. If you have already interact with packages developed through a Git (the majority of packages have a Git repository dedicate), the structure associated with it is a source package.

General, the source package is the most updated form of a package but be aware because it's mean not necessary the most stable one. To conclud, keep in our mind that this state is not associated to any OS, it's just element with a particular organisation.

6.2 - Bundled package

A bundled package is just a package compressed into a single file. By convention, R package extension is `.tar.gz`. The first one (`.tar`) is for the reduction in one single file, and the second one (`.gz`) is compressed with gzip. Like the previous state, it's again not associated to any OS and it's more a means of transportation for your package. If you are familiar with the package available on CRAN, you should see in a package description page on the CRAN website the item *Package source* with an associated `.tar.gz` file, which is the bundle version of the package.

If you decompress a bundle, you will find 3 differences with a source package structure:

- If you have vignettes (we will discuss that during the section of the documentation), they have been rendered an appear below the repositories `inst/doc/` with an index in the `build` repository.

- During the development phase, you can use temporary files for example increase the compilation time. These files are never found in a bundle.
- To finish, all the files listed into the `.Rbuildignore` of a source package is not included in the bundle because by definition there are excluded from the package compilation.

6.3 - Binary package

This next state is the first one where you have the integration of the OS specification. If you want to share your package without any package development tools mandatory for the user, you need to provide it with a binary package. Like a bundled package, a binary package is a single file but as a user you have to choose the correct one for your OS. Normally you should find two kinds of binary package extensions, one `.tgz` for macOS and another `.zip` for Windows. On Linux, you have to install tools necessary to `.tar.gz` files but some recent resource like Posit Public Package Manager provide to Linux user access to binary packages just like the other OS.

We don't go deeper into the internal structure of a binary package but be aware that is structure is very different than a source and bundled packages and if you want to go deeper you can find a very nice comparative between the three here.

6.4 - Installed package

An installed package is a binary package that's been decompressed into a package library (see section below). This is what happens when you use the function `install.packages()`. At this moment and it's practically sure that you have already some troubles in the past, it could be complicated to reinstall a package already used in the current R session. Sometime you have popups associated to "need of compilation", restart a fresh session or again question related to take a package from sources because the version is more updated that the one that you have selected. There don't have a perfect solution by when troubleshooting, Windows users should strive to install packages in a clean R session, with as few packages loaded as possible. Regarding the popup question of taking the package from the source because the version is higher, I suggest doing that only if you know what you are doing, for example if you want a specific process or function developed so far only on the source.

If you have a lot of troubles with packages installations and update, remind of the pak [9] package. We used it before to check our package name, but he provides a good and powerful alternative to `install.packages()`. Use it with caution because it is a relative newcomer, but we are certainly using it more and more in our personal workflows (in the GitHub action that we see after for example).

6.5 - In-memory package

Everyone how used R used this state all the time, thought the command `library()`. When you use that, you loaded the package associated into the memory (and for the R session) and in addition, attached the package to the search path. We will see after the difference between loading and attaching package when we talk about the "hell" of dependencies (it's a very important thing when you writing package).

To conclude this part, did not confuse the word *library* and *package*. Sometime people talk about a package as a library with for example "I use the library `DBI` to dealing with my database". A library is just like if the name suggests an element which containing installed packages, just like a library "in the world" contain books. Most of the time it is not a problem because people will understand anyway, but the distinction is important and useful as you develop package.

Part III

Second part - Package metadata

7 - The DESCRIPTION file

The file creates initially when we launch the creation of our package structure contains the major metadata information. You will find fields like the name of the package, the title and description associated, version or authors. The format of this file is DCF (Debian Control Format). It looks like the YAML on several points, each line consists of a field name and a value, separated by a colon. When you have a value display on multiple lines, like the field `Description`, the line following the first one needs to be indented (you will see that in the example below). It's a plain text file and like another one you can open it through a text editor (like NotePad or Visual Studio Code).

For our package created before, this is the initial DESCRIPTION file.

```
Package: macfly
Title: My Anwesone paCkage For Learning with Yearning
Version: 0.0.0.9000
Authors@R: c(
    person("Mathieu", "Depetris", , "mathieu.depetris@cnrs.fr", role = c("cre", "aut"),
           comment = c(ORCID = "0000-0001-8080-0531")),
    person("UMR MARBEC", "MARine Biodiversity Exploitation & Conservation", role = "cph")
  )
Description: An example of R package development with functions and all
    the stuff around to increase the joy of writing our own processes.
License: `use_mit_license()`, `use_gpl3_license()` or friends to pick a
    license
Encoding: UTF-8
Roxygen: list(markdown = TRUE)
RoxygenNote: 7.3.3
```

To update this file you have two possibilities, open the file and modify it by hand or use the R package desc [11] to manipulate it by command lines. In the following part, we will use functions of this package to reduce as possible typing error.

7.1 - Fields `Package`, `Title` and `Description`

The first line in the description file is the `Package` field and refer to the name of the package that we define before (Section 4.2). You don't have anything to modify in this field.

Like name suggest, `Title` is one line description of the package. It should be plain text (no markup), capitalized like a title, and not end in a period. Shorter is better and anyway, listings will often truncate the title to 65 characters.

`Description` field is one (and only one) paragraph more detailed than the `Title` field. You can use multiple sentences and it's recommended to use multiple lines. Each line must be no more than 80 characters wide and indent the new line with 4 spaces.

It's important to find a good title and description because it's soemthing that people see in first, especially if you submit your package on CRAN (it's display on the package CRAN website).

For our example, I used the code below. You can modify it according to our needs.

```
library(desc)
# By default the function read the DESCRIPTION file of the current working directory
description_file <- desc()
# For modify the title
description_file$set(Title = "My Anwesone paCkage For Learning with Yearning",
                     Description = "An example of R package development with functions and al
```

7.2 - Field `Version`

This field is a very important way to indicate the state of your package and evolving across the time and more precisely is lifecycle. We will go deeper into this in the [section to define].

7.3 - Field `Author`

The field `Authors@R` inside the DESCRIPTION file identify package's authors and contributors. This is a very important information because with that you know whom to contact if you are any troubles with the package. In comparison with the other field, this one contains executable R code rather than plain text. You could find also a declaration of authors like below.

```
Author: FirstName LastName [aut, cre]
```

It's the classic way but I highly recommend using the modern one with `Authors@R` because you will be able to increase the amount of information inside, like the affiliations or the ORCID number. Furthermore, it's the format recommended by CRAN. In addition, it integrate the function `person()` which hold information about individuals such as their name and email address and display them in the proper way in R.

You can use several `person()` function's arguments (take a look in the documentation to have a full overview) but you will find below the most useful and common:

- `given` and `family` specify respectively the first name and the last name. For a non-person entity, like your research unit, use only `given` and omit the `family` (except for special purpose, like if you want to have the acronym of your unit and the full name),
- `email` is very important because it's the logical way to contact for the user is he has any question on your package and processes associated. It field is in relation to the `role` below and condition if it is a mandatory information or not,
- the `role` field indicate the contribution and the implication of each author in the package development and life. In practice you can have multiple role for one person and you can choose between theses:
  - the `cre` is the creator or the maintainer of the package. You can have only one and it's the person of the entity that you have ton contact is you have troubles. In relation with comment before, this field has to be associated with a valid email. Furthermore, despite the name, this role is used for the current maintainer of the pacakge even if it's not the initial creator of the package,
  - `aut` is for authors and indicate peoples which have significant contributions to the package,
  - `ctb` is for contributors like people which made smaller contributions like patches or reviewers,
  - `cph` is copyright holder. Typically your research unity should be a copyright holder or someone who gives you code for your package.
  - Sometime you can have the `fnd` role for funder, which is like the name suggest, people or organizations that have provided financial support to the package.
- `comment` is the last one useful because it let you the opportunity to included your ORCID identifiers.

At least, you package should have one author and one maintainer (it could be the same person), with the last one associated to a valid email. In addition, you can list multiple arguments (like several roles for one person) or multiple authors with a `c()`.

According to the section before, you will find below an example for update the field `Author` of the package training exemple:

```r
# Removing default "Author" field
description_file$del("Authors@R")
# Adding authors
description_file$add_author(given = "Mathieu",
                            family = "Depetris",
                            email = "mathieu.depetris@cnrs.fr",
                            role = c("cre", "aut"),
                            orcid = "0000-0001-8080-0531")
description_file$add_author(given = "UMR MARBEC",
                            family = "MARine Biodiversity Exploitation & Conservation",
                            role = c("cph"))
```

At least, theses fields are used to generate basic citation for our package and you can use the function `citation()` for display it. You can find below the exemple of citation for the package dplyr.

```
To cite package 'dplyr' in publications use:

  Wickham H, François R, Henry L, Müller K, Vaughan D (2023). _dplyr: A
  Grammar of Data Manipulation_. doi:10.32614/CRAN.package.dplyr
  <https://doi.org/10.32614/CRAN.package.dplyr>, R package version
  1.1.4, <https://CRAN.R-project.org/package=dplyr>.

A BibTeX entry for LaTeX users is

  @Manual{,
    title = {dplyr: A Grammar of Data Manipulation},
    author = {Hadley Wickham and Romain François and Lionel Henry and Kirill Müller and Davis
    year = {2023},
    note = {R package version 1.1.4},
    url = {https://CRAN.R-project.org/package=dplyr},
    doi = {10.32614/CRAN.package.dplyr},
  }
```

So far, we don't modify any other lines in the DESCRIPTION file. The next modifications will be appear directly in the file when you will launch the r functions associated. So the last thing to do is to save the DESCRIPTION file modifications by erasing the previous one. Use the following command line do doing that.

```r
# Update, by remplacement, the DESCRIPTION file (in the current working directory)
description_file$write()
```

```
# Bonus, this function standardize the DESCRIPTION file (ordering fields and dependencies)
usethis::use_tidy_description()
```

```
Package: macfly
Title: My Anwesone paCkage For Learning with Yearning
Version: 0.0.0.9000
Authors@R: c(
    person("Mathieu", "Depetris", , "mathieu.depetris@cnrs.fr", role = c("cre", "aut"),
           comment = c(ORCID = "0000-0001-8080-0531")),
    person("UMR MARBEC", "MARine Biodiversity Exploitation & Conservation", role = "cph")
  )
Description: An example of R package development with functions and all
    the stuff around to increase the joy of writing our own processes.
License: `use_mit_license()`, `use_gpl3_license()` or friends to pick a
    license
Encoding: UTF-8
Roxygen: list(markdown = TRUE)
RoxygenNote: 7.3.3
```

7.4 - Field `License`

This field is mandatory and reflect your package's license. To simplify the thing, the license will inform users of your package what they can do, or not, when the use or manipulate your package. This is an important task and you have to ask yourself about several question to take the better descision according to your needs. You can find a deeper reflexion on this aspect in the Chapter 8.

7.5 - Field `Encoding`

If you have special characters or diacritical marks like accents in your data, you have already dealing before with the encoding, to avoid text warp. By default the field `Encoding` is UTF-8 as this is the most common encoding in use today. Normally you don't have to modify.

7.6 - Roxygen fields

By using the function `create_package()` before, two fields related to documentation will be added in the description file, `Roxygen` and `RoxygenNote`. We will see later (section to define) what they are connected to but there are in relation to the roxygen2 package, your

best ally to simplify the documenting production and icing on the cake theses fields will be updated automatically in the DESCRITPION according to actions initiate during the package development and lifecycle.

7.7 - Futures fields

You will that several new fields will be added in the DESCRIPTION file, like `URL`, `BugReports`, `Imports` or `VignetteBuilder`. We will discuss these fields in dedicated sections, but you can have an overview of them here if you are curious. For information, you can also create custom fields to add additional metadata. So far, I never have the utility to doing that but if it's relevant according to your needs, you will find some guidelines over there.

8 - Licensing

Choose your licence for your package allows you to declare how you want your code to be used and an important step if you use or include code written by someone else, to respect his work and license associated. You don't need, thankfully, to be an expert and you just need to follow several guidelines. If you have any doubt or if you are in a specific case, like if selling your package, don't hesitate to consult juridic service of your company/institute or for example a lawyer.

> **Note**
>
> In this section we will just give you basic rules to ask you the good question and if necessary go deeper in the subject. Regarding that, you will find materials like this website for open-source license comparisons, the chapter 12 of the R Packages book [1] which describes a good overview of the license topic, another book Licensing R write by Colin Fay [12] or at least the section 1.1.2 of the R manual "Writing R Extensions".

8.1 - Global overview

There are 3 main kind of licenses when you talk about software development: the open source, proprietary and source-available licenses. If we go beyond software development, you can find other families of licences in relation to the kind of creation. We can mention Creative Commons (CC) family for creative works (art, photography, music, ...), Open Data Commons (ODC) licences for data and databases licences or the GNU Free Documentation License for Documentation and Educational Content Licenses for manuals, textbooks, tutorials, and academic materials. Here we are just focusing our reflection on software development family (with a little shortcut for licences dedicated to data) because we only deal in our example with an R package.

> **Important**
>
> Keep in our mind that defines a license is an important step to do, and skip it is a possibility but if you are doing that, the default copyright laws apply: no one is allowed to make a copy of your code without your express permission.

8.2 - Software development licenses

Focusing on this kind of creation, three major families were available.

The first one is open source licences. These licenses allow users to access, modify and redistribute the source code freely. They promote transparency, collaboration, and innovation.

The second one is proprietary licences. These kind restrict access to the source code and limit how the software can be used, modified or distributed. The aims under that could be protection of commercial interests and maintain control over the software.

The last one is source-available licenses. These one make the source code visible but do not grant full open source freedoms like we could have with open source licences. This allow transparency while preserving business models or ethical boundaries.

For our use here, we mainly talk about open source licences because we will use the guideline of the FAIR principles to rule our work. We will see below how to specify a proprietary licence if necessary but I let you go deeper into these licences, and the source-available licenses, if you think your work needs one of them.

8.3 - Open source licences

In the family of open sources licences for software development we find two major kind of licences :

- The permissive licenses are the most simply to use. With this kind of license you can freely copied code, modified or published it and the only restriction is that the license must be preserved. For example, mainly of resources associated with the tidyverse field is under MIT license. If you go deeper into the question of license on tidyverse, you will find that a discussion occurred in 2021 regarding the selection of the best license for it. This brings to the table that no license if perfect and under the process of selection you have to be caution what keeps the compatibility of licenses already applied (we discuss that later in the section to define). As an example, you should also find license like Apache which is another mainly used currently permissive license.
- Copyleft licenses are stricter. One of the most popular is the General Public License, in the last version 3.0, which allows you to freely copy and modify the code for personal use, but if you publish modified versions or bundle with other code, the modified version or complete bundle must also be licensed with the GPL. With this kind of licences you ensures that software remains free and open in all its future versions.

You will find below a quick summarise of theses two kinds of licenses (table 1).

Table 1: Comparison copyleft versus permissive licenses.

Aspect	Copyleft License	Permissive License
Freedom to modify	Yes, but derivatives must keep the same license	Yes with no obligation to keep the same license
Redistribution rules	Must include source code and remain open	Can be redistributed as closed-source (associated to proprietary licences for example)
Protection of openness	Strong, prevents privatization of derivatives	Weak, allows integration into proprietary software
Commercial flexibility	Less flexible for companies	Highly flexible, widely adopted in industry
Legal complexity	More restrictive, requires careful compliance	Simple and lightweight
Philosophy	Preserve freedom for all users	Maximize adoption, even in closed environments

8.4 - Location of a R package license

When you develop a R package and you want to add a license, the information associated is located at 3 or 4 location in your source package state.

The first location is in the `DESCRIPTION` file in the `License` field (Section 7.4). This name has to be in a standard form for example pass the `R CMD check` (we will see that in the section to define). Briefly, you can have these forms :

- a name and a version specification, for example `GLP (>= 2)` or `Apache License (== 2.0)`. Related to the version, be aware of the licenses compatibility between them (check Section 8.5.1),
- a standard abbreviation, for example `GPL-2` or `MIT`,
- a name of the license "template" and a file associated (located at the root of the source package state, see the next location point). For example, in the `License` field you could find `MIT + file LICENSE`. This kind of form is only necessary for several licenses but not for all. If you apply a GPL license to your package, the mention `+ file LICENSE` is not necessary because CRAN can automaticelly identifiy the license. For simplify your life, use the usethis functions associated to each license to add them in your package (the correct form and file(s) associated will be added automatically),
- Pointer to the full text of a non-standard license (for example in the case of a proprietary licence), with `file LICENSE` in the `License` field (associated like before with a file `LICENSE`).

Hopefully, in the majority of case you will use the form `License_name_version + file LICENSE` in `License` field of the `DESCRIPTION` file but be aware that you can also have some complicated specification, with an associated with several licenses for you package (if you bundle code from another R package for example). Take a look in the section 1.1.2 of the R manual "Writing R Extensions" for concrete examples.

In relation to the first location, the second is related to the `LICENSE` file specifies before. This file could be a template required additional details to be complete (at the end you will have inside the year and copyright holder), or can also contain the full text of non-standard and non-open source licenses.

> **!** Important
>
> If your license is a standard license, you are not permitted to include the license full text associated in the `LICENSE` file.

The third location of the license information is in a `LICENSE.md` file. This file includes a copy of the full text of the license (in opposition to the `LICENSE` file of the second location where you are not allowed to include the license full text, except in the case of a non-standard or non-open source licenses). In the majority of open sources licenses, you have the obligation to include a copy of the full text license when you distribute your software. For example, in the MIT you will find the following sentence:

"The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.".

The only problem is if you want to publish your package on the CRAN, this one doesn't permit you to include a copy of standard licenses in your package. To correct that, we will use the `.Rbuildignore` file (see Section 8.7 below) to make sure this file is not sent to CRAN.

The last location of license information is optional, but could occur in a `LICENSE.note` file. We will develop this case in the Section 8.5.3 below, but just to summarize, this could happen when you bundle code in your package from other packages and you need to keep the licenses associated with the source packages.

8.5 - Application cases

Due to the difficulty to have a full overview of all licenses (it really links to your needs), tha aims of the following sections are to develop several cases related to what could occur in the development life of your package.

8.5.1 - For the code you write

In a natural way, we will start with the code that you write. It's not an exhaustive list but you should have material to make your first move. You will find at the end of the section a comparative table with usethis functions to create license resources in your package repository.

One of the most popular licenses in the world of open source license is the MIT license. This is a permissive license created in the late 1980s by the Massachusetts Institute of Technology. Globally you are allowed to use it for personal or commercial projects, modify the code however you like, distribute original or modified versions and even sell the software associated or include it in the proprietary products. Furthermore, the license is short and easy to understand, compatible with other licenses and encourages collaboration and reuse. As example, the majority of tidyverse R package were under MIT license.

Another popular open source license is the GPL (for General Public License). Created by Richard Stallman in 1989 as part of the GNU Project, unlike MIT license, GPL is a copyleft license. That mean that software, and any derivation work, remains free and open. If you distribute a modified version of the software, you must release your changes under the GPL license. You're allowed to use the software freely, modify the source code and share the original or modified version, but you can't make it proprietary. In addition, there are two major versions, GPL version 2 and GPL version 3. In summary, the GPL version 3 is an update of the version 2, more in line with the modern way. The last one is more complete, have better protection for users and developers against patent lawsuits and hardware restrictions (like tivoization or DRM/Digital Rights Management).

> **!** Important
>
> Be careful because the two licenses are not compatible (you can't bundle GPLv2 and GPLv3 code in the same project). To solve that, it's generally recommended to license your package as GPL >=2 or GPL >= 3 so that future versions of the GPL license also apply to your code.

Apache 2.0 is a permissive open source license created by the Apache Software Foundation. Similar to the MIT licence, this one is longer and more detailed and includes an explicit patent grant. Be careful because he is only compatible with GPv3, not deal with the question hardware restrictions. Basically, it gives you freedom with responsibility, ideal for businesses and developers who want flexibility without the copyleft obligations of GPL. It's a license very used in the world of Android OS.

LGPL v3 license, for Lesser General Public License, is an alternative between strict copyleft licences, like GPL, and permissive licences like MIT or Apache. Created by the Free Software Foundation, this one allow developers to use open source libraries inside proprietary software, without forcing the entire application to be open source. It's just like a weak copyleft designed mainly for software libraries. Concretely, you can use this license in proprietary or open

source applications, you are not obliged to open your code but you have to link it to the library dynamically or provide resources with the modified version. At least if you modify and distribute the library, you have to release it under the LGPL license.

The AGPL v3 (GNU Affero General Public License) is a license with a stronger copyleft. It's based on the GPl version 3 license but cover one breach of this one, using your software over a network, like SaaS or Software as a Service platforms (deployment of an R Shiny application on a web server is a SaaS). With a AGPL license, demand source code sharing even for remote use. In addition, you have to keep the software under the AGLP license if you distribute or deploy it.

If your package share only data the previous licenses not fit well because they are designed specially to apply on source code. In this case, you can take a look on two Creative Commons licences.

The first one is CC0 license for Creative Commons Zero. It's a permissive license equivalent to the MIT license, but applies to data. You don't have any restrictions, you may copy, modify, distribute, and use the work without attribution, even for commercial purposes. In addition, you don't need to cite the author (unless required by local law, as in France). Globally, the CC0 retains no rights. It is a radical alternative that aims to facilitate sharing and reuse without constraints.

The second one is the CC-BY license (the last one is the Creative Commons Attribution 4.0 International). With this one, you can copy and redistribute, but also remix, transform, and build upon the material for any purpose, even commercially. The only requirement is to give appropriate credit to the original creator, with naming the author, providing link to the license and indicating if changes were made. In clear, it's if you want to require attention when someone else uses your data.

To conclude this section, you will find below two comparatives tables to support your license choose (table 2 and table 3). In addition, you can visit the following website which give you additional support.

Table 2: Comparison between "classic" open source software development licences.

License	Type	Commercial use	Source code publication	Patent/network use protection	Hardware restrictions	Sharing licences	Other licences compatibility	usethis function
MIT	Permissive	Allowed	Not required	No/No	Not adressed	Free	Highly	`use_mit_license()`
Apache 2.0	Permissive	Allowed	Not required	Yes/No	Not adressed	Free	Compatible with GPLv3	`use_apache_license()`

License	Type	Commercial use	Source code publication	Patent/network use protection	Hardware restrictions	Sharing licences	Other licences compatibility	usethis function
LGPL v3	Weak copyleft	Allowed	Required if modified and distributed	Yes/No	Prohibited	Under LGPL or GPLv3	Compatible with GPLv3	`use_lgpl_license()`
GPL v3	Strong copyleft	Allowed	Required if distributed	Yes/No	Prohibited	Under GPLv3	Not with GPLv2, compatible with Apache 2.0, LGPL v3, AGPL v3	`use_gpl_license()`
AGPL v3	Very strong copyleft	Allowed	Required even for network use	Yes/Yes	Prohibited	Under AGPL v3	Compatible with GPLv3	`use_agpl_license()`

Table 3: Comparison between CC0 and CC-BY licences (focus on data).

Licence	Commercial use	Sharing and modification	Legal Protection	Other licences compatibility	Revocation Possible	usethis function
CC0 v1	Allowed	Free with no attribution required	Low (rights waived)	Very high	No, irreversible	`use_cc0_license()`

Licence	Commercial use	Sharing and modification	Legal Protection	Other licences compatibility	Revocation Possible	usethis function
CC-BY v4	Allowed	Free with attribution	Moderate (rights retained except attribution)	High, but attribution must be preserved	Yes, author may relicense	`use_ccby_license()`

> **Note**
>
> If you need to define a proprietary license (not open source), you can use the function `use_proprietary_license()` available in the usethis package.

8.5.2 - For the code given to you

In many case in your package life, you will include in code not written by you. The first possibility if where people inject code into your package, for example through a git pull request.

In most of cases, when someone contributes code to your package he license that content under the same terms that your package and we assume have the right to licence that content under those terms. If you use GitHub it's very clear in the GitHub terms of service page.

However, it's important to keep in mind that the author of the code retains copyright of their code, which means that you can't change the license without their permission (we discuss that in Section 8.6). If you want to keep the possibility to change the license without any needs from the author, you need an CLA (Contributor License Agreement) where the author explicitly reassigns the copyright.

In best practices it's also important to acknowledge the contributors (it's very nice to received this kind of feedback). For that, you have several options, depending of what you prefer :

- During section regarding the documentation of your package (**put section here**) we will introduce a file called `NEWS.md`. In this file, you release changes and updates of your package, and it's a good place to indicate and acknowledge the contribution to your package,
- Another option is to use the `Author` field in the `DESCRIPTION` file with the role `cph`. It could be hard at the end if you integrate all the contributors to the `DESCRIPTION` file, in terms of display of this one, but it's a possibility.

8.5.3 - For the code you bundle

The second form of integration of external code in your package could be if you bundle code. This could happen for example if you copied a piece of R code directly from another package to avoid taking dependency of it (Chapter 9). This kind of action has to be done with caution, because indeed you avoid dependency with another package (and in addition you reduce the complexity of your package and the management associated), but the code associated will is not updated automatically (for example in case of a bug fixes) and you have to deal with the licences compatibility (Section 8.5.3.1). In fact, it's really useful when you need only a very small part of the code from a big package, or if the package is no longer updated.

8.5.3.1 - License compatibility

We begin to discuss that in the table 2 and table 3, through the column "Other licences compatibility". Before bundle someone else's code into your package, you have to check licenses compatibility between your package license and the source package. Keep in your mind that you can add additional restrictions, but you can't remove restrictions. Which means that license compatibility is not symmetric.

In addition to the previous information in the tables, you can consider the following case to support you:

- If your license and their license are the same, it's OK to bundle,
- If their license is MIT or Apache, it's OK to bundle,
- If their code has a copyleft license and your code has a permissive license, you can't bundle their code (the copyleft impose on you to keep the software open source),
- If the code comes from Stack Overflow, it's licensed with the Creative Common CC BY-SA license, which is only compatible with GPLv3.

If you are none of any one of these previous cases, you have to do little research. You will find on the internet several diagrams of compatibility between licenses and you have to deal with terms like permissive compatibility, copyleft compatibility, collective work compatibility or again derivative work compatibility.

If your package isn't open source, things are more complicated and the best option could be to check with your legal department first.

8.5.3.2 - How to include in your package

If all the licenses associated to your package are compatible, you can bring the code into your package. Now it's important to follow the next steps:

- if you're including little piece of code, the best practices is to put in its own file and ensure that the file has copyright statement and license description at the top,
- if you including multiple files, put them in a directory and put a `LICENCE` file in that directory,
- you need to update the `Author` field in the `DESCRIPTION` file and used the role `cph` to declare them. In addition, you can use the `comment` field to describing what they're the author of. You could find a example here for a package named `diffviewer`,
- if you plan to submitting your package on CRAN and bundled code has different license, you have to include a `LICENSE.note` file that describes the overall license of the package and the specific licenses of each individual component. Take a look on the file from the package `diffviewer` for example.

8.5.4 - For the code you use

The last application case is directly related to the use of R. The fact is R is licensed under GPL license version 2 superior or equal.

So the question is does my code write through R has to licensing under a GPL => 2 license?

To be clear, this question has been discussed before by the R foundation and you can have a similar approch in the R package book written by Hadley Wickham and Jennifer Bryan ([1]). To really understand the subject is important to be clear of what is a redistribution or not of the R's code. This point is crucial because is you redistribution even a small portion of the R code, you have to follow the compatibility licence rule, associated to the one of R (in this case GPL). Basically, you have redistribution when you :

- copy an R function into your own package (even partially) and share your package,
- you modify an R function and redistribute it,
- you integrate R source files into your own project and share it.

When you work on R and use the R functions associated (or associated to R package), you use R as an environment (an API or Application Programming Interface). You use the system but you don't redistribute the core. In that case you can choose the license that you desire.

One concret example is for the R package ggplot2. This one uses R's base functions like `data.frame()` or `plot()`, but it does not include their source code. It interacts with R through its public API, which is allowed without triggering GPL obligations. Therefore, `ggplot2` is (and can be) licensed under MIT, even though R is under GPL, because it's not redistributing or modifying R's code.

The answer and the justification is subtle but make all the difference for your developments. You will find below a brief table to summaries the topic (table 4).

Table 4: Support for perimeter of R GPL licence obligation

Action	Redistributes R's code?	License impact
Calling R/packages functions	No	No GPL obligation
Copying/modifying R source code	Yes	Must use GPLv2 or later
Bundling R with your software	Yes	Must comply with GPLv2 or later
Writing a package that runs on R	No	Free to choose license

8.6 - Relicensing

This last point focusing on the relicensing of your package. Even if choose the best license for your purpose if better, sometime it's possible that you need to changing it during your package lifecyle. If you are in this case, and it could happen even to the big, you could use these following step to support in this work:

- first take a look into the `Author` field in the `DESCRIPTION` file and list all the contributors (with role `cph`),
- if your package is hosted on a Git, take a look at the Git history or the contributors display (for GitHub). If you have done the thing right, you could also find this information in the `NEWS.md` file,
- make a clean if necessary in the contributions and remove people who only contributed typo fixes and similar,
- remove also people whom you have a CLA (Contributor License Agreement), because you don't need their validation for the relicensing (you can inform them, of course, about the process),
- the last step is to inform all the contributors about your wails to change the license. If you have Git connection with your package, you could create an issue and make a list ask them.

When you have all the validation, you can change your license.

8.7 - Application for my package example

If you are here, that mean that you are an expert in license :)

Now we will just define a license for our R package example called "macfly". After a very hard debate in my mind, I choose to use a GPL v3 license. Behind that choose I like the idea to keep all the future production associated with my package (and there will be because my package name contains the word awesome) free and open. In the discussion the MIT license was at the

top of the list and I also think about the AGPL linked to the potential network use of my package (if I decided to add R Shiny for example), but we stay simple here. It's a joke but it is the kind of discussion that you should have and could avoid pain if you add to change your license in the middle of the life of your package.

Focusing on my want, to define a license to my package, I've to launch the following command line in the working directory of my package.

```
library(usethis)
use_gpl_license(version = 3,
                include_future = TRUE)
```

```
v Setting active project to "/home/runner/macfly".
```

In the output R console you should see something like the lines above:

- in the case of my code, if you take a look into the `DESCRIPTION` file, you should an update of the field `License` with the mention `GPL (>= 3)`. Specify argument `include_future = TRUE` create a name compatible with the future version of the license, a future version 4 of the GPL license will be compatible with the current selected version 3,
- in the package directory you have the creation of the file `LICENSE.md`. This contains the full license text. Like we say before, this is an obligation to add to your software the full license text (remember it's something specify in the license), but CRAN doesn't allow this file when you check your package regarding CRAN standard (and in addition if you want to submit it on CRAN).
- to correct that, the next added `^LICENSE\.md$` to the `.Rbuildignore`. We will know more about this file when we start to build our package (**put section here**), but the idea is to avoid specific elements when you build your package. Just to clarify, the text added is aregex expression which means at the start of the string if you find the exact word `LICENCE.md` and the end of the string after, avoid this file when you build my package.

9 - Dependencies

Dependencies and deal with them in your package is one of the most steps in your package development and in fact his lifecycle. We will see that adding a dependency could bring a lot of advantages for your processes, but come also with a portion of risk, to summarize **"great power come with great responsibilities"** and you have to define if "**I do it if it's worth it !**".

At first sight, use a maximum of dependencies could be a great idea because that mind use something already created, tested and used by others and you don't reinvent something which already exists. But introduce dependencies in your code could also have strong consequences. For example, if dependencies change and evolve over time, you maybe need to change your code and increase the maintenance associated (don't underestimate this phase because it's a lot of time consuming in your package life, especial if your package is used and share by a lot of people). In addition, add dependencies could increase the time and disk space needed when users install your package and if you have a little experience in dependency management in R you will know that topic could bring several errors during updating or installing process (in addition to the variability of each OS and user working environment).

However never use any dependency is not a good idea because even if you don't have any problem, you leave behind a lot of great potential functionalities and at the end you could increase the time of maintenance (take a look at packages which provide a process of continuous integration) and be sensitive to new bugs because you don't use the power of reflection of the global R community (you will get the benefit of all the bugs that have already been discovered and fixed).

9.1 - Global guideline

Like always, there is not one perfect way but a **good compromise** between no dependency and the hell of too many dependencies and the goal is to find your way according to your needs. To support you in the search to find the right choice for you, you will find below several advises:

- **First one**, to take into account that all the dependencies don't have the same level of consequences if you put them into your package structure (a balance between costs and benefits). For example you should have heard about the base or the stats packages. Theses kind of packages come bundled with R itself and they are very low cost of depend

on because they are nearly universally installed on all users' systems and were updated with new R versions. In opposition, if you want to add in dependencies a package from a Git repository, you should have a lot of consequences in term of maintenance and of deployment of your package to users (also associated to the potential instability to the package). Just to explain more that, you will find today a lot of packages hosted on the CRAN repository in a stable version with a development version of packages associated available on a Git (often GitHub). To be available on CRAN repository, a package must follow several rules and in particular be compatible with the all existing R environment, included packages already available on CRAN. To simplify, take a package from CRAN guarantee to have a stable version, which is not the case all the time when you deal with package hosted on a Git. Generally we take package from a Git when we want a specific function or feature, not included yet in the stable version of the package. To come back to the topic of type of dependencies, take a dependency package from CRAN will come with less of consequences that a package take from a Git (or similar) repository.

> **Note**
>
> When you update your packages in R you should have a popup menu with the question "Do you want to install from sources packages which needs compilation" with in addtion in the R console something like the image below.
> It's important to understand that if you answer yes to this question, you will install package (not in binary state) from another repository than the one defined as your package repository (like the CRAN or Bioconductor). That means that you will install the last version of the package available (in the image source version is updated compares the binary one) but maybe not the last stable version. Like we told before, if you don't need a specific feature prefer install always package from an "official" package repository.

- The **second** advice is to be careful about the recursive dependencies and the associated installation in series. For example, you can find on CRAN package with a lot of recursive dependencies and that means that these dependencies should be installed in addition to the initial package dependency. A good thing about that is that you could have some dependencies already fulfilled. For example, if your package has a dplyr dependency, adding tibble dependency does not change the dependency footprint because dplyr already depends of tibble (it's a recursive dependency). To go deeper, use packages structured in a larger community (again the case of the tidyverse packages), simplify your code because these packages could share the same behaviors regarding processes are in addition could by associated between them. In the same aspect, popular packages were probably already installed on the majority of users' machines and adding theses in dependencies should be lower in terms of consequences.

Note

Two functions of the pak package could help you to explore dependencies relations, `pkg_deps_tree()` and `pkg_deps_explain()`. Below you have an example for the dependency tree of the usethis package and the explication of the link between `usethis` and `cli` package. By default, functions give you the mandatory dependencies (we will explain that after) but you can play with the argument `dependencies = TRUE` to see all the dependencies associated (mandatory and optional) and begging to understand the "hell".

```
library(pak)
pkg_deps_tree(pkg = "usethis")
```

```
  Loading metadata database

  Loading metadata database ... done



usethis 3.2.1 [new][bld][dl] (393.91 kB)
 cli 3.6.5 [new][bld][cmp][dl] (640.24 kB)
 clipr 0.8.0 [new][bld][dl] (21.90 kB)
 crayon 1.5.3 [new][bld][dl] (40.40 kB)
 curl 7.0.0 [new][bld][cmp][dl] (731.11 kB)
 desc 1.4.3 [new][bld][dl] (80.07 kB)
   cli
   R6 2.6.1 [new][bld][dl] (64.51 kB)
 fs 1.6.6 [new][bld][cmp][dl] (1.20 MB)
 gert 2.3.0 [new][bld][cmp][dl] (129.95 kB)
   askpass 1.2.1 [new][bld][cmp][dl] (6.06 kB)
     sys 3.4.3 [new][bld][cmp][dl] (19.94 kB)
   credentials 2.0.3 [new][bld][dl] (283.14 kB)
     askpass
     curl
     jsonlite 2.0.0 [new][bld][cmp][dl] (1.06 MB)
     openssl 2.3.4 [new][bld][cmp][dl] (1.21 MB)
       askpass
     sys
   openssl
   rstudioapi 0.17.1 [new][bld][dl] (118.45 kB)
   sys
   zip 2.3.3 [new][bld][cmp][dl] (115.47 kB)
 gh 1.5.0 [new][bld][dl] (46.37 kB)
```

```
  cli
  gitcreds 0.1.2 [new][bld][dl] (62.57 kB)
  glue 1.8.0 [new][bld][cmp][dl] (126.68 kB)
  httr2 1.2.2 [new][bld][dl] (277.33 kB)
    cli
    curl
    glue
    lifecycle 1.0.5 [new][bld][dl] (107.14 kB)
      cli
      rlang 1.1.6 [new][bld][cmp][dl] (767.93 kB)
    magrittr 2.0.4 [new][bld][cmp][dl] (281.79 kB)
    openssl
    R6
    rappdirs 0.3.3 [new][bld][cmp][dl] (12.29 kB)
    rlang
    vctrs 0.6.5 [new][bld][cmp][dl] (969.07 kB)
      cli
      glue
      lifecycle
      rlang
    withr 3.0.2 [new][bld][dl] (103.24 kB)
  ini 0.3.1 [new][bld][dl] (3.49 kB)
  jsonlite
  lifecycle
  rlang
glue
jsonlite
lifecycle
purrr 1.2.0 [new][bld][cmp][dl] (278.25 kB)
  cli
  lifecycle
  magrittr
  rlang
  vctrs
rappdirs
rlang
rprojroot 2.1.1 [new][bld][dl] (59.90 kB)
rstudioapi
whisker 0.4.1 [new][bld][dl] (28.59 kB)
withr
yaml 2.3.12 [new][bld][cmp][dl] (108.97 kB)
```

```
Key:  [new] new | [dl] download | [bld] build | [cmp] compile
```

```
pkg_deps_explain(pkg = "usethis",
                 deps = "cli")
```

```
usethis -> cli
usethis -> desc -> cli
usethis -> gh -> cli
usethis -> gh -> httr2 -> cli
usethis -> gh -> httr2 -> lifecycle -> cli
usethis -> gh -> httr2 -> vctrs -> cli
usethis -> gh -> httr2 -> vctrs -> lifecycle -> cli
usethis -> gh -> lifecycle -> cli
usethis -> lifecycle -> cli
usethis -> purrr -> cli
usethis -> purrr -> lifecycle -> cli
usethis -> purrr -> vctrs -> cli
usethis -> purrr -> vctrs -> lifecycle -> cli
```

- **Third** point to be aware is the suffering and the pain of installing the package. This could be associated with the time to compile it (link for example to the complexity of the code contain), the size of the package and the worst one, the system requirements. The best example to illustrate that is the rJava package. If you have dealt with access database connection through R, you should already use it and have some troubles related to the installation and configuration of the Java SDK on your machine (especially if the user doesn't have the admin rule on his computer). So the best advice that I can get is if you can replace or not use this kind of package, do it!
- As **fourth** guideline find information about maintenance capacity of the package and globally about the communities associated. Tidyverse packages are a good example of that because you will a very large communities around them, a higher confidence associated and maintained by developers. As example, a non maintained of a dependency in your package should oblige you to modify your processes and request more maintainer from yourself.
- The **last one** is to ask yourself if you really need to use function(s) of a package to do what you want. Sometime, we just need only one function of a big thing and if we take a moment to step back maybe it's simpler to write something on your own and avoid all the "complexity" of associated a new dependence in your package. In addition, ask yourself about the potential users of your package could help: an expert on our domain should have already the same kind of packages that you asking through dependencies.

9.2 - Dependencies declarations

9.2.1 - Field in the `DESCRIPTION` file

In the section 7.7 we discussed future fields. There are two major field used for declare packages dependencies, `Imports` and `Suggests`.

`Imports` listed packages mandatory for your package work. Any time your package is installed, those packages will also be installed, if not already present. It's maybe not the best name for that process, because `Imports` doesn't means the package associated is "imported".

`Suggests` if for packages not necessary to your package work, but could be use in several particular case. It might be packages for run tests, build vignettes or development processes. In fact that mean that the regular user never use theses packages, so it's necessary to install them only if you need to.

You can find others fields for dependencies like `Depends` (used for example to define a minimum R version), `LinkingTo` (used if your package uses C or C++ code from another package) or `Enhances` but it's very specifics and you should probably never use them.

We will go deeper into that later when we define some dependencies for our package but keep in your mind that you will find at the place all the packages dependencies used in your package with some potential additional information (like the minimum version mandatory).

> **Note**
>
> It could be difficult to understand the difference between the field `Imports` and `Depends` because both will install the package dependencies when your package is installed. The difference is that when you use `Depends`, the package loaded when your package is attached. With `Imports`, you loaded the package only when you use it. Globally it's better, because with that you minimise changes to the global landscape and you look forward to the ideal goal of a package which is to be self-contained. Furthermore, it reaches the example above regarding the use of `Depends` to define à minimum R version because this "check" should be done at some kind of low levels of the process.

9.2.2 - The `NAMESPACE` file

The `NAMESPACE` file is one of the other elements created initially in the package directory. Like the name suggest, information in this file provides a location (or a space) to looking up and find an element (like a function). To summaries the aim behind that process, the idea is to **avoid any kind of confusion or mistake** related to the element that you want to manipulate. You should probably already use the concept of the `::` operator, which disambiguate functions with the same name. We will explain that deeper in the next section related to the search

path but when I say `dplyr::filter` I say to R that I want to use the `filter` function of the package `dplyr`. The `NAMESPACE` file will do exactly the same thing.

The good news is that you don't need to fill this file manually by hand because just like the first line said, this document was "Generated by roxygen2: do not edit by hand". He describe briefly the package roxygen2 in the chapter 2 but we go back to it during the chapter on documentation.

> **Note**
>
> If by mistake you make a modification by yourself in the `NAMESPACE` file instead of an automatic implementation, you should have some error with the explication of impossibility to update the file. If it's the case don't panic, just remove the file and it could be created again and updated at the next compilation of your package.

In this file, each line contains a directive which describes an R object and informs if this object is exported from this package to be used elsewhere or imported from another package to be used internally. Without being exhaustive you can find below the most common directives:

- `export()` for export a function outside the package,
- `S3method()` for export an S3 method (pseudo object),
- `importFrom()` for imported selected object form another namespace,
- `import()` for imported all objects from another package's namespace,
- `useDynLib()`, less common but for registers routines from a DLL (Dynamic Link Library).

9.2.3 - Search path

In the section below, we discuss the use of the operator `::` and the importance to leave no uncertainty or interpretation of R regarding the item that you want to use. To understand why it's dangerous, it's important to know how R work to find an element with a name. It's related to the search path. To call a function, R first has to find it. The first place to look is the global environment (the top-level environment created during a R session). If we doesn't find it there, it looks in the search path which is the list of all packages you have attached to your R session. You can display this search path by launch the code `search()` in the R console. Below, you have an example of my search path of my R session when I launch R without anything else.

```
search()
```

```
[1] ".GlobalEnv"        "package:stats"     "package:graphics"
[4] "package:grDevices" "package:utils"     "package:datasets"
[7] "package:methods"   "Autoloads"         "package:base"
```

The order here is important because in this search path each element has the next element as its parents and define the searching order. Like talk before, the first place where R locking is the global environment (`.GlobalEnv`), following by several packages, here attached automatically when you launch a R console. The item `Autoloads` is a little bit particular because it's a special environment used only when it's necessary for save memory. The last one of this search path is the base package cited before. Now, the important aspect is what happen when you attached a new package through the R console (with `library()` as example)? By doing that you modify the search path, and your package will be inserted right after the global environment. Below, I attached the pak package.

```
library(pak)
search()
```

```
 [1] ".GlobalEnv"        "package:pak"       "package:stats"
 [4] "package:graphics"  "package:grDevices" "package:utils"
 [7] "package:datasets"  "package:methods"   "Autoloads"
[10] "package:base"
```

It's a big change because that means that if the package pak and stats contains a function or an element of the same name, if I just call the element in the R console without any other indication of location, R will select the first one he finds, in this example in the package pak. It could be a serious problem if my intention was to use the function in the package stats and by doing that I create incertitude in terms of processing.

> **Note**
>
> Behind the tricky way of the search path, R provide you several messages to warning you. As an example, if you attached the `dplyr` package in your R session, you should have the message below.
>
> ```
> library(dplyr)
> ```
>
> ```
> Attaching package: 'dplyr'
>
> The following objects are masked from 'package:stats':
>
>     filter, lag
>
> The following objects are masked from 'package:base':
>
>     intersect, setdiff, setequal, union
> ```

With that message, R warning you that by attaching `dplyr` in the search path, several objects will not be selected first (in the package `stats` and `base`) if you call them in the R console with only their names (because these names were present in the `dplyr` package and "replace" the others present in packages).

! Important

In this section we discuss about packages attached to our R session but it's important to take a moment to develop this term and the subject around that, especially as you are a package developers. In the common R users way we use the function `library()` to "load" package in our R session but in fact when we doing that we attaches package to the search path and theoretically not loading it.
When you **loading** a package, you will load code, data, DLLs (little pieces of codes to simplifying) and register S3 and S4 methods and by association run the `.onLoad()` function (could be used for side-effects, for example to doing something after package loading). Your package will be available in memory but not in the search path so you won't be able to access its without using `::`. The tricky thing it's that using `::` will also load package automatically is it isn't already loaded.
In a second hand, when you **attaching** a package, you put it in the search, but you can't attach a package without loading it first, so when you use `library()` you load and attach a package (by association run `.onAttach()` function, which have the same purpose that the `.onLoad()` but when you attach package).
You will find in the table 1 below a comparative of the four functions that make a package available.
Table 1: R functions for load or attach a package.

	Return an error	Return `FALSE`
Load	`loadNamespace("package_name")` Not really used	`requireNamespace("package_name")` Used if you want a specific behaviour depending is the package is installer or not
Attach	`library(package_name)` Never use in package code below R/ or tests/, used locally if you want to terminate a script after an error	`require(package_name)` Don't trust the name and dangerous because the process will continue when in case of failure. Maybe useful in case of suggested package (see section 9.2.1)

9.2.4 - Package environment

With all the information above, you should understand the process of R to find objects regarding your needs. But to be very clear we have to talk about the package environment. It's an important aspect because you will see after when we write some code inside our package, that you have to play with this package environment to access of the process that you define in your package.

The best way to deal with that is to take a concrete example from the section 10.3.2 of the R Packages book. The function `sd()` of the `stats` pacakge use the function `var()`. But, if the user define a new function also call `var()` in the global environment, the function `sd()` will continue to use the function `var()` from the package `stats` and work correctly. This behavior is non consistent with what we expected regarding the search path. It's related to the environments.

Every function in a package is associated with a pair of environments, the package environment and the namespace environment. The package environment is the external interface to the package which exposes only objects exported. Globally, when you use the `::` operators you discuss with the package environment and you use the search path process explain above. The namespace environment is the internal interface of the package and includes all objects in the package (exported or not). That means that every link exists in the package environment also exists in the namespace environment but not to the contrary. This is why the package `stats` use the "correct" function for `var()` in our example above (a summary of the global example process is displayed in the figure 1 below).

Figure 1: Global process for R object finding (copyright [1])

9.3 - Dependencies workflow

Now you should better understand the global process behind dependencies management in R packages. In this section we will see how to implement that in our package and display a global workflow to deal with that.

Part IV

Third part - Adding code

Part V

Fourth part - Tests and checking

Part VI

Fifth part - Documentation

Part VII

Sixth part - Current management

References

1. Wickham H, Bryan J (2023) R packages: Organize, test, document, and share your code (2nd edition). O'Reilly Media

2. Wickham H, Hester J, Chang W, Bryan J (2022) Devtools: Tools to make developing r packages easier

3. Wickham H, Bryan J, Barrett M, Teucher A (2024) Usethis: Automate package and project setup

4. Wickham H, Danenberg P, Csárdi G, Eugster M (2024) roxygen2: In-line documentation for r

5. Wickham H, Hesselberth J, Salmon M, Roy O, Brüggemann S (2025) Pkgdown: Make static HTML documentation for a package

6. Wickham H (2011) Testthat: Get started with testing. The R Journal 3:5–10

7. Hester J, Angly F, Hyde R, et al (2025) Lintr: A 'linter' for r code

8. Ushey K, Wickham H (2025) Renv: Project environments

9. Csárdi G, Hester J (2025) Pak: Another approach to package installation

10. Yu G (2020) hexSticker: Create hexagon sticker in r

11. Csárdi G, Müller K, Hester J (2023) Desc: Manipulate DESCRIPTION files

12. Fay C (2019) Licensing r

A Not define yet

Appendice a